AdamNX: An Adam improvement algorithm based on a novel exponential decay mechanism for the second-order moment estimate

Meng Zhu[a], Quan Xiao[a], Weidong Min[b,c,d,*]

[a] *School of Information Management and Mathematics, Jiangxi University of Finance and Economics, Nanchang 330032, China*
[b] *School of Mathematics and Computer Science, Nanchang University, Nanchang 330031, China*
[c] *Institute of Metaverse, Nanchang University, Nanchang 330031, China*
[d] *Jiangxi Provincial Key Laboratory of Virtual Reality, Nanchang 330031, China*

Abstract

This paper studies the exponential decay mechanism of the second-moment estimate in Adam. We propose AdamNX and a time-varying second-moment decay rate that gradually weakens the correction applied to the update scale. Under the assumptions used in our analysis, this mechanism makes the updates approach momentum-SGD-like behavior during the training plateau phase. We report results on the image-classification, object-detection, and semantic-segmentation tasks, configurations, and comparators included in this paper. These results do not establish multi-seed statistical effects, flatness, or generalization beyond the reported tasks. Our code is open-sourced at `https://github.com/mengzhu0308/AdamNX`.

Keywords: Optimization algorithms, Adam, SGD, AdamNX, Exponential decay rate of second-order moment estimate.

1. Introduction

This paper focuses on Adam's second-moment estimate and its effect on update scaling, which defines the scope of the AdamNX design.

*Corresponding author
Email addresses: `zhumeng@jxufe.edu.cn` (Meng Zhu), `xiaoquan@foxmail.com` (Quan Xiao), `minweidong@ncu.edu.cn` (Weidong Min)

Table 1 chronologically outlines the milestones of optimization algorithms in deep learning. In 1986, Rumelhart *et al.* [1] first proposed an efficient algorithm to calculate the exact gradient of all weights in a multilayer network on the full dataset in one go. For large-scale datasets, the cost of iterating once with gradient descent (GD) is too high, *i.e.*, it requires too much computational resources. To address this issue, they also proposed the SGD algorithm at the same time, which randomly draws a small batch of data from the training dataset to calculate the gradient and update the model parameters. However, there are three problems for SGD: (1) Lack of "momentum"—slow convergence. Each step is taken only in the direction of the current gradient, which easily causes oscillation back and forth on the narrow "valley" shaped surface, resulting in more convergence steps. (2) Unequal learning step sizes for different parameters. When encountering sparse features or scenarios with large gradient scale differences, the direction with large gradient oscillates violently, while the direction with small gradient is almost not updated. (3) Sensitive to learning rates. A slightly larger learning rate can easily cause divergence, while a slightly smaller one can easily cause stagnation. To address problems 1 and 3 of SGD, Sutskever *et al.* [11] introduced momentum into SGD, using "inertia" to suppress oscillations and accelerate traversal through flat areas, thus making it more robust to learning rates. To address problems 2 and 3 of SGD, Tieleman *et al.* [12] proposed adaptive step sizes to combat sparse features or scenarios with large gradient scale differences, thus making it more robust to learning rates. However, both momentum SGD and RMSProp algorithms only solved partial problems in SGD, but did not simultaneously obtain the advantages of both. Therefore, Adam [2] combined momentum and adaptive learning step size, and made it more robust to learning rates through unbiased estimate, *i.e.*, bias correction. To address the problems of Adam having to cache two additional sets of parameters and high computational complexity, Chen *et al.* [13] proposed the EvoLved Sign Momentum (Lion) algorithm. Compared with Adam, Lion has lower computational complexity and is more memory-efficient. Recently, Liu *et al.* [14] proposed the Sophia algorithm, which introduces a lightweight Hessian diagonal estimate as a preconditioner and combines a clipping mechanism to control the update step size, making the pretraining speed of language models twice as fast as Adam and reducing the cost by half. Jordan *et al.* [15]

proposed the MomentUm Orthogonalized by Newton-schulz (Muon) algorithm, which is specially customized for matrix parameters.

Table 1: Key nodes and iterations of optimization algorithms in deep learning

Year	**denoteative algorithms**	**Core innovations**	**Main limitations**
1986	GD [1]	Full-data gradient, theoretically stable	The cost of iterating once on large-scale datasets is too high
1986	SGD [1]	Random batch samples for approximation, computationally efficient	(1) Lack of "momentum"—slow convergence. (2) Unequal learning step sizes for different parameters. (3) Sensitive to the learning rate
2013	momentum SGD [11]	Momentum accumulation, suppression of oscillations, and acceleration of convergence	No adaptive step size
2012	RMSProp [12]	First-order moment estimate, adaptive learning step size	Lack of directional inertia
2015	Adam [2]	Unbiased estimate of both moments	Generalization disadvantage, additional storage required for first-order and second-order moments
2017	AdamW [3]	Decoupling of weight decay and target updates	Better generalization than Adam, but still requires additional storage for first-order and second-order moments
2023	Lion [13]	Sign-based Momentum	Not as effective as Adam in small batches
2024	Sophia [14]	Introduction of lightweight diagonal Hessian estimate, combined with a clipping mechanism to control the update step size	(1) Information loss due to diagonal approximation. (2) Hard trade-off between estimate frequency and timeliness
2024	Muon [15]	Updates for matrix parameters	Higher computational cost than Adam, increased parallel communication cost

Adam maintains first- and second-moment estimates and uses the latter to scale updates. Prior work has discussed different late-stage update characteristics of adaptive and momentum methods. We propose AdamNX (see Algorithm 1), which gradually weakens this correction through a time-varying second-moment decay rate. Under the assumptions of our analysis, its plateau-phase updates approach momentum-SGD-like behavior. We do not present this mechanism as a proof of flatness, generalization, or general stability.

The remainder of this paper is organized as follows. Section 2 reviews the existing research on optimization algorithms in deep learning and introduces the motivation

of this study. Section 3 provides a detailed analysis of the problem and proposes the corresponding solution. Sections 4, 5, and 6 jointly verify the superiority of AdamNX through experiments. The final section summarizes the content of the entire paper.

2. Related work

2.1. Optimization algorithms in deep learning

In 1986, Rumelhart *et al.* [1] first proposed the backpropagation algorithm that can calculate the precise gradient of a multilayer network on the entire training set in one go. Although this algorithm is theoretically rigorous, when facing large-scale datasets, the computational and storage costs of a single iteration become a bottleneck. To alleviate this problem, Rumelhart *et al.* [1] also proposed to use mini-batch gradient approximation to replace the full gradient in SGD. The core idea is to randomly select a small batch of samples to estimate the gradient in each iteration, thereby significantly reducing the cost of a single parameter update. However, the convergence behavior of SGD in practice is not ideal, mainly manifested in three aspects of defects: (1) Lack of momentum. The update direction completely depends on the current gradient, resulting in frequent oscillations on narrow or high-curvature error surfaces and slow convergence. (2) Inconsistent step size. When the gradient scales of different parameters vary greatly, a uniform learning rate cannot take care of both sparse and dense features at the same time, resulting in some parameters being stagnant in updates and others vibrating violently. (3) Sensitive to learning rate. A slightly larger learning rate can easily cause the objective function to diverge, while a smaller one falls into inefficient updates.

To address defects 1 and 3 of SGD, Sutskever *et al.* [11] introduced an exponentially weighted momentum term into SGD, integrating historical gradient information into the current direction to suppress oscillations and accelerate traversal of flat regions. Meanwhile, the robustness to the learning rate is also improved. Gitman *et al.* [18] used stochastic differential equations to characterize the effects of different momentum values on escaping saddle points and convergence speed, and provided a practical range for momentum selection. Ramezani-Kebrya *et al.* [19] provided upper bounds

on the stability and generalization error of momentum SGD, theoretically explaining why momentum not only accelerates training but also reduces the risk of overfitting. Zhao *et al.* [20] proposed the "normalization + momentum" combination, proving that it can maintain an $O(1/T)$ convergence efficiency under anisotropic noise, effectively alleviating the directional oscillations of SGD.

To address defects 2 and 3 of SGD, Duchi *et al.* [21] proposed the AdaGrad algorithm, which achieves per-parameter adaptive learning step size by accumulating the sum of squares of historical gradients, effectively improving the convergence efficiency in sparse data scenarios. However, the accumulation of the sum of squares of historical gradients as the denominator in AdaGrad causes the learning step size to monotonically decrease to close to zero during the training process, and updates almost stop in the later stages. To address this issue, Tieleman *et al.* [12] proposed RMSProp, which replaces the global accumulation with the exponential moving average of the gradient squares to avoid the learning step size from dropping to zero.

Although momentum SGD and RMSProp each improved some local problems of SGD, their advantages were not obtained at the same time. Kingma *et al.* [2] proposed Adam, which integrates the momentum mechanism and adaptive step size strategy, and further introduces bias correction to offset the systematic bias of the exponentially weighted average, thereby further improving the robustness to the learning rate. Reddi *et al.* [22] proposed the AMSGrad algorithm, which replaces the second-order momentum sliding average in Adam with the maximum value of historical gradients to avoid converging to local extreme solutions with poor generalization. Loshchilov *et al.* [3] proposed AdamW, which decouples weight decay from the target update of the optimization algorithm, making it an independent regularization term from the target update, improving generalization ability, and making hyperparameter tuning more robust. Shazeer *et al.* [23] proposed the Adafactor algorithm, which achieves sublinear memory consumption through low-rank decomposition of the second-order moment, effectively reducing the training memory requirements and is suitable for large model training. Liu *et al.* [14] proposed Sophia, which introduces lightweight diagonal Hessian estimation into large-scale language model pretraining, achieving faster convergence and lower computational cost than Adam. Sophia uses diagonal Hessian approx-

imation instead of the complete Hessian matrix, which reduces the computational cost but sacrifices the curvature correlation information between parameters. The Hessian is estimated every k steps, which keeps the average computational cost within 5% of the gradient calculation, but may lead to dynamic response lag. The estimation variance may be amplified in the early training stage or in small-batch scenarios, affecting stability.

AdamNX differs structurally from SWATS's discrete switch [16] and AdaI's dynamical modification [17]: it uses a continuous time-varying second-moment coefficient within an Adam-style update. We retain this structural comparison but do not report experiments against SWATS or AdaI.

In addition, Adam and AdamW need to store the first-order moment estimate and the second-order moment estimate for each parameter, which have high memory occupation and computational complexity. To address the high memory and high computation defects, Chen *et al.* [13] proposed Lion, which effectively reduces memory demand and computational volume through symbolic momentum, and can match or even outperform Adam in most tasks. However, when the batch size is small (less than 64), Lion is not as good as Adam. Recently, Jordan *et al.* [15] proposed Muon for matrix parameters, which uses Newton-Schulz iteration to approximately orthogonalize the momentum update matrix, significantly improving the training efficiency of matrix parameters. However, when the matrix is split and distributed across multiple devices, Muon must first aggregate the gradients of each slice (all-reduce) and then calculate the update amount uniformly, which cannot be updated independently in parallel on each device, thus introducing additional communication overhead.

In summary, from full gradient to SGD, and then to momentum, adaptive step sizes, bias correction, and specialized optimizers for structural features, the evolution of optimization algorithms in deep learning has always revolved around "improving convergence efficiency—reducing computational costs—enhancing hyperparameter robustness".

2.2. Research motivation

We consider a continuous second-moment decay schedule. It retains stronger adaptive correction early in training and weakens it later. Under the stated assumptions, it illustrates an Adam-style update approaching momentum-SGD-like behavior. It does not establish algorithmic degeneration or gains in flatness or generalization.

3. Proposed algorithm

This section analyzes the effect of β_2 on the variance of Adam's second-moment estimate under explicit assumptions, then introduces a time-varying second-moment decay rate and AdamNX. The analysis offers a mechanism-oriented explanation only; it does not provide a convergence rate, regret bound, or general stability guarantee. We then compare the second-moment coefficients of AdamNX and Adam.

3.1. Impact of β_2 on the variance of the second-order moment estimation in Adam

Let:

$$\boldsymbol{g}_t = \nabla f(\boldsymbol{\theta}_{t-1}) + \boldsymbol{\xi}_t \tag{1}$$

where $\nabla f(\boldsymbol{\theta}_{t-1})$ denotes the full-batch gradient, *i.e.*, the gradient of $\boldsymbol{\theta}_{t-1}$ computed using all samples $\mathcal{D}$, $\boldsymbol{g}_t$ denotes the stochastic gradient, *i.e.*, the gradient of $\boldsymbol{\theta}_{t-1}$ computed by randomly sampling a subset $\mathcal{R} \subset \mathcal{D}$ from $\mathcal{D}$, and $\boldsymbol{\xi}_t$ denotes the noise introduced by random sampling. Thus, $\boldsymbol{g}_t$ is an approximation to $\nabla f(\boldsymbol{\theta}_{t-1})$. To close the following derivation, we use idealized isotropy, independence, and i.i.d. assumptions. In practical mini-batch training, noise may depend on the parameters, data augmentation, and time, so these assumptions are usually approximations. The following results are used as a heuristic analysis under Hypotheses 1, 2, and 3:

Hypothesis 1. *The full-batch gradient vector $\nabla f(\boldsymbol{\theta}_{t-1})$ and the noise vector $\boldsymbol{\xi}_t$ have independent and identically distributed (i.i.d.) components across dimensions.* [1]

Hypothesis 2. $\boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{1})$.

[1] This assumption was implicitly used in the derivations of arXiv v1 but was omitted from the formal statement of assumptions.

Hypothesis 3. *The full-batch gradient $\nabla f(\boldsymbol{\theta}_{t-1})$ and the stochastic noise $\boldsymbol{\xi}_t$ are mutually independent.*

Based on Hypothesis 2, Equation (1) can be written as:

$$\boldsymbol{g}_t = \nabla f(\boldsymbol{\theta}_{t-1}) + \sigma\boldsymbol{\xi} \tag{2}$$

where $\boldsymbol{\xi}$ denotes the standard normal distribution.

For Adam, the second moment estimate is given by Equation (3):

$$\begin{cases} \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2, \\ \widehat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t} \end{cases} \tag{3}$$

where $\boldsymbol{g}_t$ denotes the gradient of the current iteration, $\boldsymbol{v}_{t-1}$ denotes the second moment of historical iterations. Iterating Equation (3), we have:

$$\boldsymbol{v}_t = (1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}\boldsymbol{g}_i^2 \tag{4}$$

Substituting Equation (2) into Equation (4) yields:

$$\begin{aligned} \boldsymbol{v}_t &= (1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}(\nabla f(\boldsymbol{\theta}_{i-1}) + \sigma\boldsymbol{\xi})^2 \\ &= (1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}((\nabla f(\boldsymbol{\theta}_{i-1}))^2 + 2\nabla f(\boldsymbol{\theta}_{i-1}) \odot \sigma\boldsymbol{\xi} + \sigma^2\boldsymbol{\xi}^2) \end{aligned} \tag{5}$$

Computing the expectation of the second moment estimate:

$$\mathbb{E}[\boldsymbol{v}_t] = (1-\beta_2^t)\sigma^2\mathbf{1} + (1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}\mathbb{E}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right] \tag{6}$$

Consider the bias term:

$$\mathbb{E}[\widehat{\boldsymbol{v}}_t] = \mathbb{E}\left[\frac{\boldsymbol{v}_t}{1-\beta_2^t}\right] = \sigma^2\mathbf{1} + \frac{1-\beta_2}{1-\beta_2^t}\sum_{i=1}^{t}\beta_2^{t-i}\mathbb{E}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right] \tag{7}$$

Computing the variance of the second moment estimate:

$$\begin{aligned} \mathrm{Var}[\boldsymbol{v}_t] &= \mathbb{E}[\boldsymbol{v}_t^2] - \mathbb{E}[\boldsymbol{v}_t]^2 \\ &= (1-\beta_2)^2\sum_{i=1}^{t}\beta_2^{2(t-i)}\left(2\sigma^4\mathbf{1} + 4\sigma^2\mathbb{E}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right] + \mathrm{Var}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right]\right) \end{aligned} \tag{8}$$

Consider the bias term:

$$\mathrm{Var}[\widehat{\boldsymbol{v}}_t] = \frac{\mathrm{Var}[\boldsymbol{v}_t]}{(1-\beta_2^t)^2} \tag{9}$$

Then, when $t \to \infty$, $\mathrm{Var}[\widehat{\boldsymbol{v}}_t]$ converges to:

$$\lim_{t\to\infty} \mathrm{Var}[\widehat{\boldsymbol{v}}_t] = \frac{1-\beta_2}{1+\beta_2}\left(2\sigma^4\mathbf{1} + 4\sigma^2\mathbb{E}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right] + \mathrm{Var}\left[(\nabla f(\boldsymbol{\theta}_{i-1}))^2\right]\right) \tag{10}$$

Under these assumptions, Equation (10) shows that the variance of the second-moment estimate decreases as β_2 increases during the plateau phase (*i.e.*, $t \to \infty$); as $\beta_2 \to 1$, this variance term tends to zero. This explains why a larger β_2 can reduce variation in the update scale, but it does not by itself prove that Adam strictly degenerates into momentum SGD. A very large β_2 also makes the second-moment estimate slower to respond to changes in the gradient distribution, and its bias correction approaches its steady state more slowly. Thus, β_2 trades smoothing against responsiveness. This is a mechanism explanation under the stated assumptions, not a nonconvex convergence-rate, regret, or general-stability result.

3.2. Novel exponential decay rate for the second-order moment estimation and AdamNX

For clarity, β_1 and β_2 are constant base decay rates, while $\widehat{\beta}_{1,t}$ and $\widehat{\beta}_{2,t}$ are time-varying coefficients. The symbol $\boldsymbol{v}_t$ denotes the raw second-moment estimate and $\widehat{\boldsymbol{v}}_t$ its bias-corrected form. We consider a β_2 that varies with iteration t, with a corresponding adjustment of β_1, and has the following properties:

(1) $\widehat{\beta}_{2,t=1} = 0$, $\widehat{\beta}_{2,t\to\infty} = 1$.

(2) For all $t \geq 1$, having: $\widehat{\beta}_{2,t} \geq \widehat{\beta}_{1,t}$.

(3) During the early and middle stages of training, $\widehat{\beta}_{2,t}$ is as close as possible to $\widehat{\beta}_{1,t}$.

Therefore, this paper proposes a novel decay rate for the second moment estimate:

$$\widehat{\beta}_{2,t} = \frac{1-\beta_2^{(1-\beta_2)(t-1)}}{1-\beta_2^{(1-\beta_2)t}} \tag{11}$$

It is easy to prove that:

$$\lim_{t\to\infty} \widehat{\beta}_{2,t} = \lim_{t\to\infty} \frac{1-\beta_2^{(1-\beta_2)(t-1)}}{1-\beta_2^{(1-\beta_2)t}} = 1 \tag{12}$$

and correspondingly proposes the AdamNX algorithm, as shown in Algorithm 1. To align with Adam's default settings: $\beta_1 = 0.9, \beta_2 = 0.999$, AdamNX is set to: $\beta_1 = 0.9$, $\beta_2 = 0.99$. This allows hyperparameters tuned for Adam to be better transferred to AdamNX. Figure 1 visualizes $\widehat{\beta}_{1,t}$ and $\widehat{\beta}_{2,t}$ in AdamNX.

Algorithm 1: AdamNX algorithm

Input$_1$: training dataset $\mathcal{D}$.
Input$_2$: parameters to be optimized $(\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_L)$, first-order moments $(\boldsymbol{m}_1, \boldsymbol{m}_2, \ldots, \boldsymbol{m}_L)$, second-order moments $(\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_L)$.
Input$_3$: learning rate η_t, weight decay rate λ, exponential decay rate $(\beta_1, \beta_2) = (0.9, 0.99)$, $\epsilon = 1 \times 10^{-8}$, total number of iterations T.
Output: extremum $(\boldsymbol{\theta}_1^*, \boldsymbol{\theta}_2^*, \ldots, \boldsymbol{\theta}_L^*)$.

for $t = 1$ **to** T **do**
  randomly drawing a subset of training samples: $\mathcal{R}_t \subseteq \mathcal{D}$;
  /* forward propagation */
  loss value calculation: $\mathcal{L}_t = \frac{1}{|\mathcal{R}_t|} \sum_{(\boldsymbol{x}_i, \boldsymbol{y}_i) \in \mathcal{R}_t} E(\widehat{\boldsymbol{y}}(\boldsymbol{x}_i, \boldsymbol{\theta}_{1,t-1}, \ldots, \boldsymbol{\theta}_{L,t-1}), \boldsymbol{y}_i)$;
  /* backpropagation */
  for $l = 1$ **to** L **do**
    gradient calculation: $\boldsymbol{g}_{l,t} = \nabla_{\boldsymbol{\theta}_{l,t-1}} \mathcal{L}$;
    first-order moment estimate: $\boldsymbol{m}_{l,t} = \frac{\beta_1 - \beta_1^t}{1 - \beta_1^t} \boldsymbol{m}_{l,t-1} + \left(1 - \frac{\beta_1 - \beta_1^t}{1 - \beta_1^t}\right) \boldsymbol{g}_{l,t}$;
    second-order moment estimate: $\boldsymbol{v}_{l,t} = \frac{1 - \beta_2^{(1-\beta_2)(t-1)}}{1 - \beta_2^{(1-\beta_2)t}} \boldsymbol{v}_{l,t-1} + \left(1 - \frac{1 - \beta_2^{(1-\beta_2)(t-1)}}{1 - \beta_2^{(1-\beta_2)t}}\right) \boldsymbol{g}_{l,t}^2$;
    gradient element normalization: $\boldsymbol{u}_{l,t} = \frac{\boldsymbol{m}_{l,t}}{\sqrt{\boldsymbol{v}_{l,t}} + \epsilon}$;
    parameter updating: $\boldsymbol{\theta}_{l,t} = \boldsymbol{\theta}_{l,t-1} - \eta_t \left(\boldsymbol{u}_{l,t} + \lambda_l \boldsymbol{\theta}_{l,t-1}\right)$, $\lambda_l = \begin{cases} \lambda, & \text{if } \boldsymbol{\theta}_l \text{ is a matrix parameter} \\ 0, & \text{if } \boldsymbol{\theta}_l \text{ is not a matrix parameter} \end{cases}$;
  /* saving the optimal loss value and extremum point */
  if $\mathcal{L}_t < \mathcal{L}_0$ **then**
    $\mathcal{L}_0 \leftarrow \mathcal{L}_t$;
    for $l = 1$ **to** L **do**
      $\boldsymbol{\theta}_l^* \leftarrow \boldsymbol{\theta}_{l,t}$; // storing the extremum point

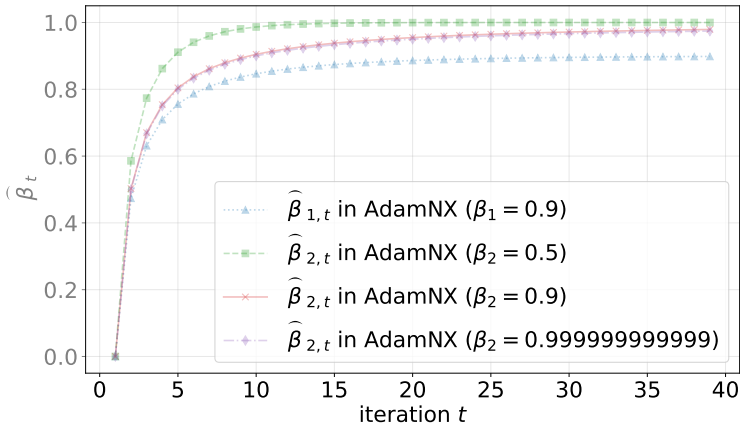


Figure 1: Functional curves of $\widehat{\beta}_{1,t}$ and $\widehat{\beta}_{2,t}$ versus iteration number t in AdamNX

3.3. AdamNX v.s. *Adam*

In fact, the recurrence of Adam's bias-corrected $\widehat{\boldsymbol{v}}_t$ also implicitly contains $\widehat{\beta}_{2,t}$:

$$\begin{aligned}\widehat{\boldsymbol{v}}_t &= \frac{\boldsymbol{v}_t}{1-\beta_2^t} = \frac{\beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2}{1-\beta_2^t} = \frac{(\beta_2 - \beta_2^t)\widehat{\boldsymbol{v}}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2}{1-\beta_2^t} \\ &= \frac{\beta_2 - \beta_2^t}{1-\beta_2^t}\widehat{\boldsymbol{v}}_{t-1} + \left(1 - \frac{\beta_2 - \beta_2^t}{1-\beta_2^t}\right)\boldsymbol{g}_t^2 \triangleq \widehat{\beta}_{2,t}\boldsymbol{v}_{t-1} + (1-\widehat{\beta}_{2,t})\boldsymbol{g}_t^2\end{aligned} \tag{13}$$

And it is easy to prove:

$$\lim_{t\to\infty} \frac{\beta_2 - \beta_2^t}{1-\beta_2^t} = \beta_2 \tag{14}$$

Comparing Equations (12) and (14), Adam's corresponding coefficient tends to β_2 as $t \to \infty$, whereas the AdamNX coefficient tends to 1. Within the recurrence interpretation and assumptions used here, this makes AdamNX updates more momentum-SGD-like late in training. This describes a mechanism trend rather than strict degeneration or general stability. Figure 2 further visualizes the comparison between AdamNX's $\widehat{\beta}_{2,t}$ and Adam's $\widehat{\beta}_{2,t}$ for different β_2 values.

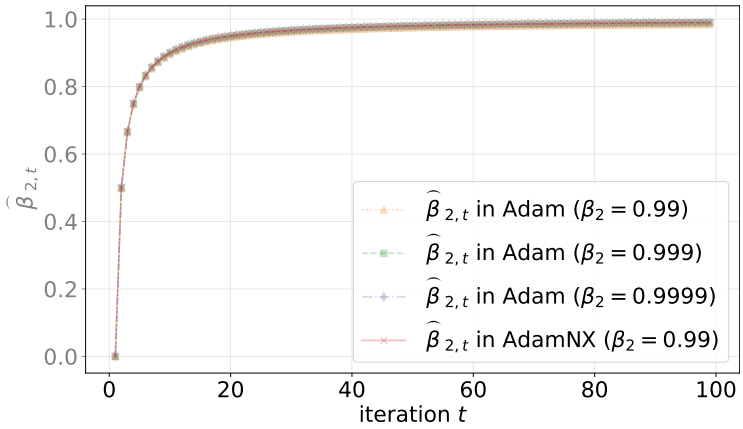


Figure 2: Comparison between AdamNX's $\widehat{\beta}_{2,t}$ and Adam's $\widehat{\beta}_{2,t}$

4. Experimental reproduction details

This section introduces the common experimental details for Sections 5 and 6. We report single results for the visual tasks, configurations, and comparators available in this paper. We do not report multi-seed means, standard deviations, significance tests, Hessian or sharpness measurements, timing or memory measurements, a unified tuning budget, or non-visual tasks. The results below therefore describe comparisons within these reported configurations and do not support claims of statistical optimality, engineering efficiency, or cross-task generalization.

Table 2 shows the main software and hardware configurations required for the experiments.

Table 2: Main software and hardware configurations

Torch [26]	2.8.0
Torchvision [26]	0.23.0
Python	3.13.7
NVIDIA CUDA Toolkit	12.9
NVIDIA cuDNN	10.2
Memory Capacity	192 GB
CPU	Intel(R) Core i9-14900KF
GPU	NVIDIA RTX A5000

The learning rate strategy configuration. The experiments in the following sections followed two different learning rate strategies depending on the model. One was a fixed learning rate, and the other followed the learning rate strategy shown in Equation (15):

$$\eta_t = \begin{cases} \eta_{\text{peak}} + \frac{\eta_{\text{min}} - \eta_{\text{peak}}}{t_1} t, & \text{if } 0 \leq t \leq t_1, \\ \eta_{\text{min}}, & \text{if } t > t_1 \end{cases} \tag{15}$$

where η_{min} denotes the minimum learning rate, and η_{peak} denotes the peak learning rate. Figure 3 visualizes the functional relationship between the learning rate and the number of iterations.

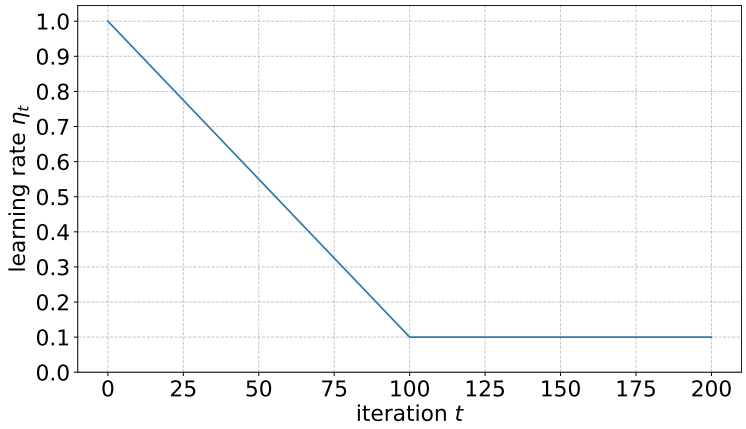


Figure 3: Functional curve between the learning rate and the number of iterations

Note: The settings of the horizontal and vertical coordinates in the figure are only for intuitively presenting the trend of the learning rate changing with the iteration process, and the values do not reflect the actual data in the experiments.

5. Comparison results and analysis of optimization algorithms of the same type

This section compares AdamNX with Adam [2] and AdaX [25]. The experiments cover CIFAR-100 [27] for image classification, PASCAL VOC [28] for object detection, and Semantic Boundaries [29] for semantic segmentation, using EfficientNetV2 [30], SwinV2 [31], YOLOv7-tiny [32], and U-Net [33]. These comparisons do not include AdamW, SWATS, or AdaI.

5.1. Image classification experimental results and analysis

Benchmark dataset for image classification. CIFAR-100 [27] is a benchmark image classification dataset constructed by the Department of Computer Science at the University of Toronto in 2009. As an extended version of the CIFAR series, it contains $60,000$ RGB color images with 32×32 pixel, forming a 100-class fine-grained classification system with 20 superclasses (each containing 5 subclasses). This dataset is strictly divided into $50,000$ training images and $10,000$ test images. During training, data augmentation strategies including random color space transformation and random horizontal flipping were employed.

Benchmark models for image classification. EfficientNetV2 [30] is a convolutional neural network (CNN) series proposed by Google. Its core innovation lies in the compound scaling method, which uniformly adjusts the network depth, width, and input resolution to achieve the optimal balance between model efficiency and performance. SwinV2[31] is an improved version of the visual Transformer proposed by Microsoft, with several innovations aimed at large-scale model training. To match the image resolution in CIFAR, EfficientNetV2 retained only the last three downsamplings; in SwinV2, the patch size was set to 1, and the window size was set to 4. The reproduction of all models directly cloned the official implementation of torchvision, and model weight initialization followed the official default without loading pre-trained weights.

Evaluation metrics for image classification. Top-1 error rate and Top-5 error rate are mainstream metrics for measuring the performance of image classification models. Suppose the test set contains N images, where the true label of the i-th image is y_i, and the model outputs a class probability vector for this image denoted as $p_i = (p_{i,1}, p_{i,2}, \ldots, p_{i,C})$ (C is the total number of classes), then the Top-1 error rate and Top-5 error rate are defined as:

$$E_{\text{Top-1}} = \frac{1}{N} \sum_{i=1}^{N} \chi_A \left(y_i \neq \arg\max_j p_{i,j} \right) \tag{16}$$

$$E_{\text{Top-5}} = \frac{1}{N} \sum_{i=1}^{N} \chi_A \left(y_i \notin \text{top-5} \left(p_{i,j}{}_{j=1}^{C} \right) \right) \tag{17}$$

where $\chi_A(.)$ denotes the indicator function. The above definitions follow the convention of reference [34] and are widely used in subsequent studies to unify the evaluation

criteria.

Lowest error rate comparison. Table 3 shows the comparative experimental results on the benchmark model EfficientNetV2-M. The total number of training epochs was set to 80, the batch size was set to 64, the peak learning rate was set to 3×10^{-3}, the minimum learning rate was set to 3×10^{-5}, and the training t_1 was set to $37,488$. As seen in Table 3, our AdamNX achieved the lowest top-1 error rate of 34.27% and the lowest top-5 error rate of 11.40% on EfficientNetV2-M. Table 4 shows the comparative experimental results on the benchmark model SwinV2-S. The total number of training epochs was set to 80, the batch size was set to 64, and the fixed learning rate was set to 2.2×10^{-5}. As seen in Table 4, our AdamNX also achieved the lowest top-1 error rate of 39.42% and the lowest top-5 error rate of 14.68% on SwinV2-S.

Table 3: Image classification results on CIFAR-100 for optimization algorithms of the same type (benchmark model is EfficientNetV2-M)

	Top-1 err. (%) ↓	Top-5 err. (%) ↓
Adam [2]	39.44	14.76
AdaX [25]	36.35	12.99
AdamNX (Ours)	**34.27**	**11.40**

Note: the **bold** denotes the best results in each column.

Table 4: Image classification results on CIFAR-100 for optimization algorithms of the same type (benchmark model is SwinV2-S)

	Top-1 err. (%) ↓	Top-5 err. (%) ↓
Adam [2]	40.05	14.80
AdaX [25]	39.97	15.07
AdamNX (Ours)	**39.42**	**14.68**

5.2. Object detection experimental results and analysis

Object detection benchmark datasets. Following the standard protocol in the object detection field of PASCAL VOC [28]: the trainval subsets of VOC2007 and VOC2012 were merged (a total of $16,551$ images containing 20 classes of objects) as the training data, and the publicly annotated VOC2007 test ($4,952$ images) was used for model evaluation. During training, data augmentation strategies including random padding

and scaling, random color space transformation, and random horizontal flipping were employed.

Benchmark model for object detection. YOLOv7-tiny was the high-speed lightweight version of the YOLOv7 [32] series. It achieved millisecond-level inference while maintaining high detection accuracy through a streamlined ELAN-Tiny backbone network, balancing detection accuracy and efficiency. The image resolution fed into the model was set to 320×320. Model weight initialization followed the initialization strategy of RegNet [35].

Evaluation metrics for object detection. Mean Average Precision (mAP) is the most commonly used evaluation metric in the field of object detection, and its calculation process is as follows. For each class j, all prediction boxes are sorted in descending order of confidence and matched with the ground-truth boxes one by one. If IoU $\geq T$ (*e.g.*, $T = 0.5$ or 0.75), and the classes are consistent, it is recorded as a True Positive (TP); otherwise, it is a False Positive (FP). Precision and recall are cumulatively calculated as follows:

$$P_j(k) = \frac{\sum_{i=1}^{k} \mathrm{TP}_i}{k} \tag{18}$$

$$R_j(k) = \frac{\sum_{i=1}^{k} \mathrm{TP}_i}{N_j} \tag{19}$$

where N_j denotes the total number of ground-truth boxes for class j, and k denotes the top k prediction boxes. The $P_j(R_j)$ curve is plotted, and the area under the curve is taken to obtain the average precision (AP) for this class:

$$\mathrm{AP}_j = \int_0^1 P_j(R_j) \mathrm{d}R_j \tag{20}$$

The mean value over all classes is then calculated as: $\mathrm{mAP} = \frac{1}{C} \sum_{j=1}^{C} \mathrm{AP}_j$.

Highest mAP comparison. Table 5 shows the comparative experimental results on the VOC2007 test benchmark dataset. The total number of training epochs was set to 80, the batch size was set to 32, the peak learning rate was set to 3×10^{-3}, the minimum learning rate was set to 3×10^{-5}, and the training t_1 was set to $25,680$. As seen in Table 5, our AdamNX achieved the highest mAP@0.5 of 52.18% and the highest mAP@0.75 of 28.21%.

Table 5: Object detection results on VOC2007 test for optimization algorithms of the same type

	mAP@0.5 (%) ↑	mAP@0.75 (%) ↑
Adam [2]	52.07	27.45
AdaX [25]	51.99	27.19
AdamNX (Ours)	**52.18**	**28.21**

Note: mAP@0.5 denotes IoU $\geq$ 0.5, and mAP@0.75 denotes IoU $\geq$ 0.75.

5.3. Semantic segmentation experimental results and analysis

Benchmark dataset for semantic segmentation. The Semantic Boundaries [29] dataset contains 11, 355 images from the PASCAL VOC 2011 dataset, divided into 8, 498 training images and 2, 857 validation images, providing pixel-level semantic segmentation annotations and object boundary information for 21 classes of objects, supporting semantic segmentation and boundary detection tasks. During training, data augmentation strategies including random scaling, random horizontal flipping, and random cropping were employed.

Benchmark model for semantic segmentation. U-Net [33] was initially proposed for biomedical image segmentation. Its symmetric encoder-decoder structure integrates high-resolution details and low-resolution semantic information through skip connections, enabling pixel-level accurate prediction with limited annotated data. Given its design that balances localization accuracy and context modeling capabilities, U-Net has become a widely adopted benchmark model in the field of semantic segmentation. The parameter basic units in U-Net were set to 32, and the image resolution fed into the model is set to 320×320. Model weight initialization followed the initialization strategy of RegNet [35]. The loss function employed a weighted average combination of cross-entropy loss [1] and Dice loss [36].

Evaluation metric for semantic segmentation. Mean Intersection over Union (mIoU) is the most commonly used evaluation metric in the field of semantic segmentation, and its calculation method is as follows: for each class, the intersection over union between the prediction results and the ground truth is calculated and then averaged:

$$\text{mIoU} = \frac{1}{C} \sum_{i=1}^{C} \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i + \text{FN}_i} \tag{21}$$

where C denotes the total number of classes, and TP_i, FP_i, FN_i denote the number of TP, FP, and false negative pixels for class i, respectively. This metric measures both localization accuracy and classification accuracy simultaneously, and a higher value indicates better segmentation performance.

Highest mIoU comparison. Table 6 shows the comparative experimental results on the Semantic Boundaries benchmark dataset. The total number of training epochs was set to 80, the batch size was set to 32, the peak learning rate was set to 1×10^{-3}, the minimum learning rate was set to 1×10^{-5}, and the training t_1 was set to $12,720$. As can be seen from Table 6, our AdamNX achieved the highest mIoU of 37.81%.

Table 6: Semantic segmentation results on Semantic Boundaries for optimization algorithms of the same type

	mIoU (%) ↑
Adam [2]	34.43
AdaX [25]	37.23
AdamNX (Ours)	**37.81**

5.4. Ablation study results and analysis

Figure 4 visualizes the second-order moment estimate exponential decay rates in different optimization algorithms. The ablation experiments in this section focused on comparing the different second-order moment estimate exponential decay rates shown in this figure. Therefore, the $\widehat{\beta}_{2,t}$ in Adam was replaced with the $\widehat{\beta}_{2,t}$ from Adafactor [23], AdaX, and AdamNX, respectively, to verify which second-order moment estimate exponential decay rate is superior.

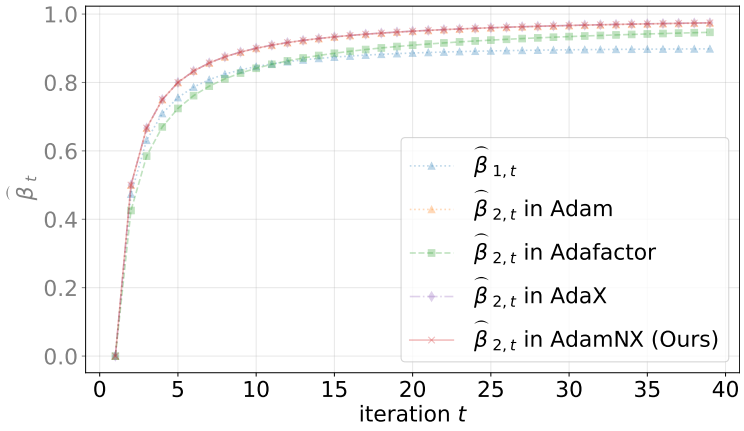


Figure 4: Different second-order moment estimate exponential decay rates

Reported-result comparison. Table 7 shows the ablation results for EfficientNetV2-M on CIFAR-100. Table 8 reports the corresponding results for SwinV2-S, Table 9 for YOLOv7-tiny on VOC2007 test, and Table 10 for U-Net on Semantic Boundaries. The AdamNX coefficient obtains the tabulated values in these configurations. Single reported results do not establish statistical optimality; the following are analytic properties of the coefficient, not properties demonstrated by these experiments:

(1) $\widehat{\beta}_{2,t=1} = 0, \widehat{\beta}_{2,t\to\infty} = 1.$

(2) For all $t \geq 1$, it holds that: $\widehat{\beta}_{2,t} \geq \widehat{\beta}_{1,t}$.

Table 7: Image classification ablation experimental results for second-order moment estimate exponential decay rates (benchmark model is EfficientNetV2-M)

Adam[2] + $\widehat{\beta}_{2,t}$	Top-1 err. (%) ↓	Top-5 err. (%) ↓
$(\beta_2 - \beta_2^t)/(1 - \beta_2^t), \beta_2 = 0.999$ (Adam [2])	39.44	14.76
$1 - 1/t^c, c = 0.8$ (Adafactor [23])	41.22	15.23
$1 - \beta_2/((1 + \beta_2)^t - 1), \beta_2 = 0.0001$ (AdaX [25])	35.22	11.92
$\left(1 - \beta_2^{(1-\beta_2)(t-1)}\right) / \left(1 - \beta_2^{(1-\beta_2)t}\right), \beta_2 = 0.99$ (Ours)	**34.27**	**11.40**

Table 8: Image classification ablation experimental results for second-order moment estimate exponential decay rates (benchmark model is SwinV2-S)

Adam[2] + $\widehat{\beta}_{2,t}$	Top-1 err. (%) ↓	Top-5 err. (%) ↓
$(\beta_2 - \beta_2^t)/(1-\beta_2^t), \beta_2 = 0.999$ (Adam [2])	40.05	14.80
$1 - 1/t^c, c = 0.8$ (Adafactor [23])	40.64	15.05
$1 - \beta_2/((1+\beta_2)^t - 1), \beta_2 = 0.0001$ (AdaX [25])	39.64	**14.50**
$\left(1-\beta_2^{(1-\beta_2)(t-1)}\right)/\left(1-\beta_2^{(1-\beta_2)t}\right), \beta_2 = 0.99$ (Ours)	**39.42**	<u>14.68</u>

Note: the <u>underline</u> indicates the second-best results in each column.

Table 9: Object detection ablation experimental results for second-order moment estimate exponential decay rates

Adam[2] + $\widehat{\beta}_{2,t}$	mAP@0.5 (%) ↑	mAP@0.75 (%) ↑
$(\beta_2 - \beta_2^t)/(1-\beta_2^t), \beta_2 = 0.999$ (Adam [2])	52.07	27.45
$1 - 1/t^c, c = 0.8$ (Adafactor [23])	45.50	21.63
$1 - \beta_2/((1+\beta_2)^t - 1), \beta_2 = 0.0001$ (AdaX [25])	51.74	27.24
$\left(1-\beta_2^{(1-\beta_2)(t-1)}\right)/\left(1-\beta_2^{(1-\beta_2)t}\right), \beta_2 = 0.99$ (Ours)	**52.18**	**28.21**

Table 10: Semantic segmentation ablation experimental results for second-order moment estimate exponential decay rates

Adam[2] + $\widehat{\beta}_{2,t}$	mIoU (%) ↑
$(\beta_2 - \beta_2^t)/(1-\beta_2^t), \beta_2 = 0.999$ (Adam [2])	34.43
$1 - 1/t^c, c = 0.8$ (Adafactor [23])	31.76
$1 - \beta_2/((1+\beta_2)^t - 1), \beta_2 = 0.0001$ (AdaX [25])	36.37
$\left(1-\beta_2^{(1-\beta_2)(t-1)}\right)/\left(1-\beta_2^{(1-\beta_2)t}\right), \beta_2 = 0.99$ (Ours)	**37.81**

Comparison of image classification training curves. The experimental logs from Table 8 were visualized to plot the training curves, which are shown in Figure 5. When plotting the function curves, downsampling and exponential weighted smoothing (seen Appendix A for the specific code) were used to more intuitively compare the different convergence characteristics due to the different second-order moment estimate exponential decay rates. As seen in Figure 5, when the number of iterations was approximately less than $19,344$, the convergence rates of the four are similar. When the number of iterations was approximately greater than $19,344$, the $\widehat{\beta}_{2,t}$ of AdaX and our $\widehat{\beta}_{2,t}$ converged faster, indicating that $\widehat{\beta}_{2,t\to\infty} \to 1$ helps to accelerate convergence in the later stages of training.

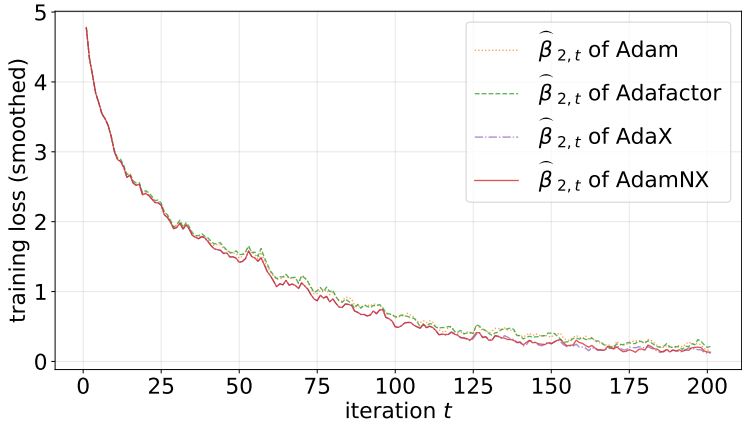
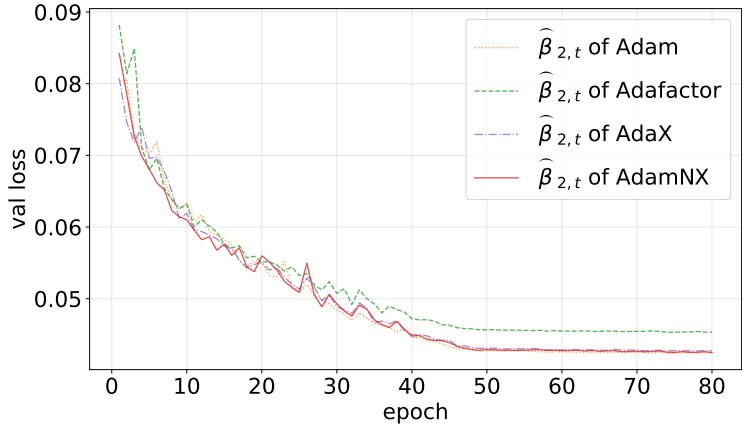
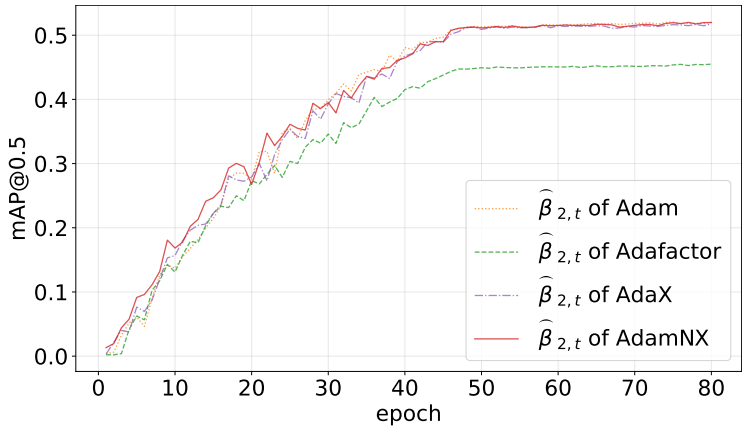


Figure 5: Training loss *vs.* iteration (experimental logs from Table 8)

Comparison of object detection evaluation curves. The experimental logs from Table 9 were plotted as curves to further analyze the convergence characteristics of different optimization algorithms, and the results are shown in Figure 6. As seen in this figure, throughout the entire training process, the $\widehat{\beta}_{2,t}$ of Adam, the $\widehat{\beta}_{2,t}$ of AdaX, and the $\widehat{\beta}_{2,t}$ of this paper all converged faster and better than the $\widehat{\beta}_{2,t}$ of Adafactor, indicating the effectiveness of "for all $t \geq 1$, it holds that: $\widehat{\beta}_{2,t} \geq \widehat{\beta}_{1,t}$".



(a) Training loss *vs.* epoch



(b) mAP@0.5 *vs.* epoch

Figure 6: Evaluation curve comparison

6. Comparison results and analysis of optimization algorithms of different type

This section compares AdamNX with momentum SGD [11], RAdam [24], Lion [13], and SophiaG [14]. The total number of training epochs was 80 and the batch size was 64. Each method uses the configuration reported here. Systematic tuning and multi-seed statistics were not used. Therefore, the comparison is limited to these configurations.

Table 11 shows the comparative experimental results on the benchmark model EfficientNetV2-L. For momentum SGD, the momentum was set to 0.9, the peak learn-

ing rate was set to 0.1, the minimum learning rate was set to 1×10^{-3}, t_1 was set to $37,488$, and the weight decay rate was set to 1×10^{-3}; for Lion, the peak learning rate was set to 1×10^{-3}, the minimum learning rate was set to 1×10^{-5}, t_1 was set to $37,488$, and the weight decay rate was set to 0.1; for RAdam, SophiaG, and AdamNX, the peak learning rate was set to 0.01, the minimum learning rate was set to 1×10^{-4}, t_1 was set to $37,488$, and the weight decay rate was set to 0.01. As seen in Table 11, our AdamNX achieved the lowest top-1 error rate of 35.66% and the lowest top-5 error rate of 12.37% on EfficientNetV2-L.

Table 11: Image classification results on CIFAR-100 for different types of optimization algorithms (benchmark model is EfficientNetV2-L)

	Top-1 err. (%) ↓	Top-5 err. (%) ↓
momentum SGD [11]	39.03	16.19
RAdam [24]	36.12	12.66
Lion [13]	41.33	17.03
SophiaG [14]	41.01	15.42
AdamNX (Ours)	**35.66**	**12.37**

Table 12 shows the comparative experimental results with the baseline model being ConvNeXt-B. To adapt to the image resolution in CIFAR, ConvNeXt-B only retained the last three downsamplings, all convolutional kernel sizes were set to $(3, 3)$, and the padding was set to $(1, 1)$. For momentum SGD, the momentum was set to 0.9, the peak learning rate was set to 0.01, the minimum learning rate was set to 1×10^{-4}, t_1 was set to $37,488$, and the weight decay rate was set to 1×10^{-3}; for Lion, the peak learning rate was set to 1×10^{-4}, the minimum learning rate was set to 1×10^{-6}, t_1 was set to $37,488$, and the weight decay rate was set to 0.1; for RAdam, SophiaG, and AdamNX, the peak learning rate was set to 1×10^{-3}, the minimum learning rate was set to 1×10^{-5}, t_1 was set to $37,488$, and the weight decay rate was set to 0.01. As shown in Table 11, our AdamNX achieved the lowest top-1 error rate of 32.87% and the lowest top-5 error rate of 10.72% on ConvNeXt-B.

Table 12: Image classification results on CIFAR-100 for different types of optimization algorithms (benchmark model is ConvNeXt-B)

	Top-1 err. (%) ↓	Top-5 err. (%) ↓
momentum SGD [11]	63.83	33.66
RAdam[24]	35.45	11.95
Lion[13]	35.60	12.54
SophiaG[14]	37.39	13.48
AdamNX (Ours)	**32.87**	**10.72**

7. Conclusion

This paper proposes AdamNX and the time-varying second-moment coefficient $\widehat{\beta}_{2,t} = \left(1-\beta_2^{(1-\beta_2)(t-1)}\right) / \left(1-\beta_2^{(1-\beta_2)t}\right)$. Under the assumptions used in our analysis, the coefficient gradually weakens update-scale correction and makes plateau-phase updates approach momentum-SGD-like behavior. The reported visual-task results show the tabulated values for AdamNX in the listed configurations and against the listed comparators. This paper does not establish convergence rates, regret bounds, general stability, flatness, statistical significance, engineering overhead, or cross-task generalization; the mathematical correctness of the relevant recurrence remains subject to author verification.

Acknowledgments

This work was partly supported by the National Natural Science Foundation of China (Grant No. 62076117) and the Jiangxi Provincial Key Laboratory of Virtual Reality (Grant No. 2024SSY03151).

References

[1] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536. doi:10.1038/323533a0.

[2] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Proceedings of the International Conference on Learning Representations, 2015, pp. 1–15.

[3] I. Loshchilov, F. Hutter, Fixing weight decay regularization in adam (2017).
URL http://arxiv.org/abs/1711.05101

[4] I. E. Parlak, Blockchain-assisted explainable decision traces (baxdt): An approach for transparency and accountability in artificial intelligence systems, Knowledge-Based Systems 329 (2025) 1–17. doi:10.1016/j.knosys.2025.114402.

[5] S. Koziel, A. Pietrenko-Dabrowska, M. Wojcikowski, B. Pankiewicz, On memory-based precise calibration of cost-efficient no2 sensor using artificial intelligence and global response correction, Knowledge-Based Systems 290 (2024) 1–17. doi:10.1016/j.knosys.2024.111564.

[6] K. Guo, M. Wu, Z. Soo, et al., Artificial intelligence-driven biomedical genomics, Knowledge-Based Systems 279 (2023) 1–16. doi:10.1016/j.knosys.2023.110937.

[7] H. Touvron, L. Martin, K. Stone, et al., Llama 2: Open foundation and fine-tuned chat models (2023).
URL https://arxiv.org/abs/2307.09288

[8] J. Jumper, R. Evans, A. Pritzel, et al., Highly accurate protein structure prediction with alphafold, Nature 596 (7873) (2021) 583–589. doi:10.1038/s41586-021-03819-2.

[9] A. Merchant, S. Batzner, S. S. Schoenholz, et al., Scaling deep learning for materials discovery, Nature 624 (7990) (2023) 80–85. doi:10.1038/s41586-023-06735-9.

[10] S. K. Kim, R. Shousha, S. M. Yang, et al., Highest fusion performance without harmful edge energy bursts in tokamak, Nature Communications 15 (1) (2024) 3990–4001. doi:10.1038/s41467-024-48415-w.

[11] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: Proceedings of the International Conference on Machine Learning, 2013, p. 1139–1147.

[12] T. Tieleman, G. Hinton, Rmsprop: Divide the gradient by a running average of its recent magnitude (2012).
URL https://cir.nii.ac.jp/crid/1370017282431050757

[13] X. Chen, C. Liang, D. Huang, et al., Symbolic discovery of optimization algorithms, in: Proceedings of the International Conference on Neural Information Processing Systems, 2023, pp. 1–30.

[14] H. Liu, Z. Li, D. L. W. Hall, et al., Sophia: A scalable stochastic second-order optimizer for language model pre-training, in: Proceedings of the International Conference on Learning Representations, 2024, pp. 1–30.

[15] K. Jordan, Y. Jin, V. Boza, et al., Muon: An optimizer for hidden layers in neural networks (2024).
URL https://kellerjordan.github.io/posts/muon

[16] N. S. Keskar, R. R. Socher, Improving generalization performance by switching from adam to sgd (2017).
URL http://arxiv.org/abs/1712.07628

[17] Z. Xie, X. Wang, H. Zhang, et al., Adaptive inertia: Disentangling the effects of adaptive learning rate and momentum, in: Proceedings of the International Conference on Machine Learning, Vol. 162, 2022, pp. 24430–24459.

[18] I. Gitman, H. Lang, P. Zhang, L. Xiao, Understanding the role of momentum in stochastic gradient methods, in: Proceedings of the International Conference on Neural Information Processing Systems, Vol. 32, 2019, pp. 1–11.

[19] A. Ramezani-Kebrya, K. Antonakopoulos, V. Cevher, et al., On the generalization of stochastic gradient descent with momentum, Journal of Machine Learning Research 25 (22) (2024) 1–56.

[20] S. Zhao, C. Shi, Y. Xie, W. Li, Stochastic normalized gradient descent with momentum for large-batch training, SCIENCE CHINA Information Sciences 67 (11) (2024) 1–15. doi:10.1007/s11432-022-3892-8.

[21] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, The Journal of Machine Learning Research 12 (2011) 2121–2159.

[22] S. J. Reddi, S. Kale, S. Kumar, On the convergence of adam and beyond, in: Proceedings of the International Conference on Learning Representations, 2018, pp. 1–23.

[23] N. Shazeer, M. Stern, Adafactor: Adaptive learning rates with sublinear memory cost, in: Proceedings of the International Conference on Machine Learning, Vol. 80, 2018, pp. 4596–4604.

[24] L. Liu, H. Jiang, P. He, et al., On the variance of the adaptive learning rate and beyond, in: Proceedings of the International Conference on Learning Representations, 2020, pp. 1–13.

[25] W. Li, Z. Zhang, X. Wang, P. Luo, Adax: Adaptive gradient descent with exponential long term memory (2020).
URL https://openreview.net/forum?id=r1l-5pEtDr

[26] A. Paszke, S. Gross, F. Massa, et al., Pytorch: An imperative style, high-performance deep learning library, in: Proceedings of the International Conference on Neural Information Processing Systems, Vol. 32, 2019, pp. 1–12.

[27] A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images (2009).
URL https://citeseerx.ist.psu.edu

[28] M. Everingham, J. Winn, The pascal visual object classes challenge 2012 (voc2012) development kit, Pattern Analysis, Statistical Modelling and Computational Learning, Tech. Rep 8.

[29] B. Hariharan, P. Arbeláez, L. Bourdev, et al., Semantic contours from inverse detectors, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2011, pp. 991–998. doi:10.1109/ICCV.2011.6126343.

[30] M. Tan, Q. Le, Efficientnetv2: Smaller models and faster training, in: Proceedings of the International Conference on Machine Learning, Vol. 139, 2021, pp. 10096–10106.

[31] Z. Liu, H. Hu, Y. Lin, et al., Swin transformer v2: Scaling up capacity and resolution, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 12009–12019.

[32] C. Wang, A. Bochkovskiy, H. Liao, Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 7464–7475. doi:10.1109/CVPR52729.2023.00721.

[33] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Proceedings of the Medical Image Computing and Computer-Assisted Intervention, 2015, pp. 234–241.

[34] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: Proceedings of the International Conference on Neural Information Processing Systems, Vol. 25, 2012, pp. 1–9.

[35] I. Radosavovic, R. P. Kosaraju, R. Girshick, et al., Designing network design spaces, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 10428–10436.

[36] F. Milletari, N. Navab, S. A. Ahmadi, V-net: Fully convolutional neural networks for volumetric medical image segmentation, in: Proceedings of the IEEE/CVF International Conference on 3D Vision, 2016, pp. 565–571. doi:10.1109/3DV.2016.79.

Appendix A. Downsampling and exponentially weighted smoothing code

```
import pandas as pd
from matplotlib import pyplot as plt
from pathlib import Path
```

```
4
5  from plotting_config import *
6  from som_decay_rate_config import *
7
8  log_dir = Path("./excels")
9  T = 62480
10 step = max(1, T // 400)
11 window = max(5, step * 0.02)
12 alpha = 2 / (window + 1)
13
14 plt.rcParams.update(plt_update)
15 fig, ax = plt.subplots(figsize=figsize)
16
17 for i, file in enumerate(log_dir.glob("*.xlsx")):
18     df = pd.read_excel(file)
19
20     ds = df.iloc[::step]
21     smooth = ds['loss'].ewm(alpha=alpha).mean()
22     file_stem = file.stem.split(" ")[-1]
23     label = r"$\overset{\frown}{\beta}_{2,t}$" + " of " + file_stem
24     plt.plot(range(1, len(smooth)+1), smooth, linewidth=1.2, alpha=0.8,
           label=label,
25                     linestyle=linestyles[file_stem], color=colors[file_stem
                       ])
26
27
28 plt.xlabel("iteration $t$")
29
30 plt.ylabel('training loss (smoothed)')
31 plt.grid(alpha=0.3)
32 plt.legend()
33 plt.tight_layout()
34
35 save_path = f"pdfs/som-decay-rate-train-loss-vs-iter-step{step}.pdf"
36 fig.savefig(save_path, bbox_inches='tight', pad_inches=0)
37
38 plt.close()
```